

Format for describing dependencies of source files

Ben Boeckel, Brad King

<ben.boeckel@kitware.com, brad.king@kitware.com>

version P1689R1,

Table of Contents

[1. Abstract](#)

[2. Changes](#)

[2.1. R1 \(post-Cologne\)](#)

[2.2. R0 \(Initial\)](#)

[3. Introduction](#)

[4. Format](#)

[4.1. Schema](#)

[4.2. Storing binary data](#)

[4.3. Filepaths](#)

[4.4. Source items](#)

[4.5. Dependency information](#)

[4.6. Future dependency information](#)

[4.7. Extensions](#)

[5. Versioning](#)

[6. References](#)

Document number ISO/IEC/JTC1/SC22/WG21/P1689R1

Date

Reply-to Ben Boeckel, Brad King, ben.boeckel@kitware.com, brad.king@kitware.com

Audience EWG (Evolution), SG15 (Tooling)

1. Abstract

When building C++ source code, build tools need to discover dependencies of source files based on their contents. This must be done during the build because the contents of the files can change without the build tools themselves rerunning. In addition, generated source files must have their dependencies discovered during the build as well. With the advent of modules in [\[P1103R3\]](#), there are now ordering requirements among compilation rules. These also need to be discovered during the build. This paper specifies a format for communicating this information to build tools.

2. Changes

2.1. R1 (post-Cologne)

The following changes have been made in response to feedback in SG15:

- rename keys to be more "noun-like" or for clarity including:
- `readable` → `readable-name`
- `logical` → `logical-name`
- `filepath` → `compiled-module-path`
- remove `future-link` (no known use case)
- remove %-encoding for filepaths
- remove top-level `extensions` key (still possible, just use `_` keys)
- require vendor prefixes for extensions
- add an optional `source-path` key to `depinfo` objects

The following changes have been made in response to feedback in SG16:

- change the name of the "data" key to "code-units"
- mention normalization for JSON encoders and decoders

2.2. R0 (Initial)

Description of the format and its semantics.

3. Introduction

This paper describes a format primarily for use during the build of C++ source code to communicate dependencies of a source file. Other uses may exist, but its primary use case is for correct compilation of C++ sources. The tool which generates this format is referred to as a "dependency scanning tool" in this paper.

This information includes:

- the dependencies of running the dependency scanning tool itself;
- the resources that will be required to exist when the scanned translation unit is compiled; and
- the resources that will be provided when the scanned translation unit is compiled.

This information is sufficient to allow a build tool to order compilation rules to get a valid build in the presence of C++ modules.

4. Format

The format uses JSON [\[ECMA-404\]](#) as a base for encoding its information. This is suitable because it is structured (versus a plain-text format), parsers for JSON are readily available (versus candidates with a custom structural format), and the format is simple to implement (versus candidates such as YAML or TOML).

JSON specifies that documents are Unicode. However, due to the way filepaths are represented in this format, it is further constrained to be a valid UTF-8 sequence.

4.1. Schema

For the information provided by the format, the following JSON Schema [\[JSON-Schema\]](#) may be used.

► JSON Schema for the format

4.2. Storing binary data

This format uses UTF-8 as a communication channel between a dependency scanning tool and a build tool, but filepath encodings are specific to the platform in use. Therefore, considerations for paths containing non-UTF-8 sequences must be made. However, the most common uses of paths and filenames are either valid UTF-8 sequences or may be unambiguously represented using UTF-8 (e.g., a platform using UTF-16 for its path APIs has a valid UTF-8 encoding), so requiring excessive obfuscation in all cases is unnecessary.

In order to store a non-UTF-8 sequence losslessly, there must be a way to encode the non-UTF-8 sequence into this format. There have been multiple ways utilized in the past for storing binary data into JSON including Base64 (as well as other related encodings such as Base85 or Base91), integer arrays, and going so far as to convert the entire file format over to binary (e.g., [\[BSON\]](#), [\[UBJSON\]](#), etc.). These encodings do not handle sequences of 16-bit data well either since endianness information is not stored in them. These solutions are over-pessimistic about the common case of valid UTF-8 paths used in this format so this encoding scheme uses UTF-8 wherever possible while dropping down to a less efficient encoding only when necessary.

Note that some JSON encoders and decoders will normalize Unicode sequences. Due to the presence of platforms where non-normalized sequences are valid paths, any such normalization logic should be disabled when interacting with this format.

The most general format for storing data is to use an array of integers tagged with the size of the values in memory. This is done by using an object with two required keys: `code-units` storing the integers representing the raw data and `format` describing the size of the integers in memory. Supported formats are `raw8` and `raw16`. Other formats are ill-formed. There is an optional `readable-name` key which contains a string for communicating the contents in a human-readable format using UTF-8. The value of the `readable-name` key is purely information and does not have any normative meaning to the interpretation of the format.

- `raw8` indicates that the integers of the `code-units` array are 8-bit unsigned integers. All values of the `code-units` array are required to be integers in the range of 1 to 255, inclusive.

Example `raw8`-encoded filepath

```
{
  "format": "raw8",
  "code-units": [112, 97, 197, 163, 104, 45, 116, 111, 45, 102, 105, 108, 195, 171],
  "readable-name": "path-to-file"
}
```

- **raw16** indicates that the integers of the **code-units** array are 16-bit unsigned integers. All values of the **code-units** array are required to be integers in the range of 1 to 65535, inclusive.

Example raw16-encoded filepath

```
{  
  "format": "raw16",  
  "code-units": [112, 97, 355, 104, 45, 116, 111, 45, 102, 105, 108, 235],  
  "readable-name": "paṯh-to-fiḷë"  
}
```

Requirements for passing data to the platform's APIs such as a terminating ASCII **NUL** byte or endianness are not included in the format. Using integer values outside of the range specified for the format is ill-formed.

Example filepaths represented as UTF-8 strings

```
[  
  "paṯh-to-fiḷë",  
  "path-to-file-ascii",  
]
```

When a path can be communicated as a series of UTF-8 codepoints, it should be done, but it is not required. That is, all fields which may contain binary data in the format are allowed to be unconditionally encoded using the most general format.

4.3. Filepaths

Filepaths may either be relative or absolute. It is preferred to use relative paths because the compilation may occur in a different working directory than the scanning tool uses. However, any paths which are not dependent on the working directory of the tool must be output using an absolute path. To this end, the dependency scanning tool must output its working directory in the **work-directory** key at the root of the document. The build tool may then construct the absolute paths as necessary.

For concrete examples where absolute paths may not be suitable:

- A distributed build may perform the compilation in a different directory on another machine than the host machine is using.
- A build tool may use a chroot for each command it invokes.^[1]

4.4. Source items

The `sources` array allows for the dependency information of multiple files to be specified in a single file. The only restriction placed on this is that the `input` field across all `sources` entries be unique after decoding it as a filepath.

4.5. Dependency information

Each source represented in the `sources` array is a JSON object which only requires a single key, `input`. Its value is a `datablock` representing a filepath. Two optional keys exist to indicate the dependencies of the execution of the dependency scanning tool: the `outputs` array and the `depends` array. The `outputs` array in which each element is a filepath for files written by the dependency scanning tool due to the specified `input` file. The `depends` array in which each element is a filepath for files which affect the results of the run. For C++, this will generally paths be due to `#include`, but other mechanisms may be in effect.

4.6. Future dependency information

The core of this specification is the `future-compile` key on a `sources` object. They both use the same specification for their values, but contain the information for different phases of source compilation. These JSON objects have three optional keys, `outputs`, `provides`, and `requires`.

The `outputs` array contains filepaths which will be written to when the source is compiled. Only filepaths which are known to the dependency scanning tool that will be created at compile time should be included here.

The `provides` and `requires` arrays contain descriptions of modules that will be provided or required at compile time. Each item of these arrays is a JSON object with one required key, `logical-name`, and two optional keys: `compiled-module-path` and `source-path`. All of these key's values are filepaths. The `logical-name` value is what build tools should use to discover the ordering among translation unit compilations. In C++, this will generally be the name of the module (including its partition, if any) as included in the source. The `compiled-module-path` should be provided only if the location of the module's future on-disk representation is known when the dependency information is discovered. The `source-path` is the path to the main source of the module. This is intended to be used to communicate the on-disk header for a header-unit import when it is known.

Example source entry with future-compile information

```
{
  "input": "path/to/input.cxx",
  "future-compile": [
    {
      "outputs": [
        "path/to/output.o"
      ],
      "provides": [
        {
          "compiled-module-path": "exported.bmi",
```

```

        "logical-name": "exported"
    }
],
"requires": [
    {
        "logical-name": "imported"
    }
]
]
}

```

4.7. Extensions

Extensions may be added to the format using keys prefixed with an underscore (`_`) followed by a vendor-specific string followed by another underscore. None of these may be used to store semantically relevant information required to execute a correct build. Essentially, consumers of the format may ignore all `_`-prefixed keys and not suffer any loss of essential functionality.

Example source entry with extended information

```

{
    "input": "path/to/input",
    "_VENDOR_extension": true
}

```

5. Versioning

There are two keys with integer values in the top-level JSON object of the format: `version` and `revision`. The `version` key is required and if `revision` is not provided, it can be assumed to be `0`. These indicate the version of the information available in the format itself and what features may be used. Tools creating this format should have a way to create older versions and revisions of the format to support consumers that do not support the newer versions.

The `version` integer is incremented when semantic information is different than a previous version. This is information that is required for a build to be correct. When the `version` is incremented, the `revision` integer is reset to `0`.

The `revision` integer is incremented when the semantic information of the format is the same as previous revisions of the same version, but it may include additionally specified information or use an

additionally specified format for the same information. For example, adding a `format` type would cause an increment of the `revision`.

The version specified in this document is:

Version fields for this specification

```
{  
  
  "version": 1,  
  
  "revision": 0  
  
}
```

6. References

- [BSON] BSON (Binary JSON) Serialization. <http://bsonspec.org/>.
- [ECMA-404] The JSON Data Interchange Syntax. <http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf>.
- [JSON-Schema] Austin Wright and Henry Andrews. JSON Schema: A Media Type for Describing JSON Documents. <https://tools.ietf.org/html/draft-handrews-json-schema-01>.
- [P1103R3] Richard Smith. Merging Modules. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1103r3.pdf>.
- [RFC3629] Francois Yergeau. UTF-8, a transformation format of ISO 10646. <https://tools.ietf.org/html/rfc3629>.
- [Unicode-12] Unicode Consortium. Unicode 12.0.0. <https://www.unicode.org/versions/Unicode12.0.0/>.
- [UBJSON] Universal Binary JSON. <http://ubjson.org/>.

¹. Concretely, the Tup build tool can execute compile rules inside of individual FUSE chroots where absolute paths are meaningless outside of that context.